



FIGURE 11.53 (a) Building image of size 600×600 pixels. (b) Image smoothed using a 5×5 box kernel. (c) and (d) MSERs detected using $T = 0$, $\Delta T = 10$, and MSER size range between 10,000 and 30,000 pixels, corresponding approximately to 3% and 8% of the area of the image. (e) Composite image.

of the original area, we reduced the valid MSER range by one-fourth to 2500–7500 pixels. Other than these changes, we used the same parameters as in Fig. 11.53. Figure 11.55(c) shows the resulting MSER. As you can see, this figure is quite close to the full-size result in Fig. 11.53(e).

11.7 SCALE-INVARIANT FEATURE TRANSFORM (SIFT)

SIFT is an algorithm developed by Lowe [2004] for extracting invariant features from an image. It is called a *transform* because it transforms image data into scale-invariant coordinates relative to local image features. SIFT is by far the most complex feature detection and description approach we discuss in this chapter.

As you progress through this section, you will notice the use of a significant number of experimentally determined parameters. Thus, unlike most of the formulations of individual approaches we have discussed thus far, SIFT is strongly heuristic. This is a consequence of the fact that our current knowledge is insufficient to tell us how



FIGURE 11.54 (a) Building image rotated 5° counterclockwise. (b) Smoothed image using the same kernel as in Fig. 11.53(b). (c) Composite MSER detected using the same parameters we used to obtain Fig. 11.53(e). The MSERs of the original and rotated images are almost identical.

to assemble a set of reasonably well-understood individual methods into a “system” capable of addressing problems that cannot be solved by any single known method acting alone. Thus, we are forced to determine experimentally the interplay between the various parameters controlling the performance of more complex systems.

When images are similar in nature (same scale, similar orientation, etc), corner detection and MSERs are suitable as whole image features. However, in the presence of variables such as scale changes, rotation, changes in illumination, and changes in viewpoint, we are forced to use methods like SIFT.

SIFT features (called *keypoints*) are invariant to image scale and rotation, and are robust across a range of affine distortions, changes in 3-D viewpoint, noise, and changes of illumination. The input to SIFT is an image. Its output is an n -dimensional feature vector whose elements are the invariant feature descriptors. We begin our discussion by analyzing how scale invariance is achieved by SIFT.



FIGURE 11.55 (a) Building image reduced to half-size. (b) Image smoothed with a 3×3 box kernel. (c) Composite MSER obtained with the same parameters as Fig. 11.53(e), but using a valid MSER region size range of 2,500–7,500 pixels.

SCALE SPACE

The first stage of the SIFT algorithm is to find image locations that are invariant to scale change. This is achieved by searching for stable features across all possible scales, using a function of scale known as *scale space*, which is a multi-scale representation suitable for handling image structures at different scales in a consistent manner. The idea is to have a formalism for handling the fact that objects in unconstrained scenes will appear in different ways, depending on the scale at which images are captured. Because these scales may not be known beforehand, a reasonable approach is to work with all relevant scales simultaneously. Scale space represents an image as a one-parameter *family* of smoothed images, with the objective of simulating the loss of detail that would occur as the scale of an image decreases. The parameter controlling the smoothing is referred to as the *scale parameter*.

In SIFT, Gaussian kernels are used to implement smoothing, so the scale parameter is the standard deviation. The reason for using Gaussian kernels is based on work performed by Lindberg [1994], who showed that the only smoothing kernel that meets a set of important constraints, such as linearity and shift-invariance, is the Gaussian lowpass kernel. Based on this, the scale space, $L(x, y, \sigma)$, of a grayscale image, $f(x, y)$,[†] is produced by convolving f with a variable-scale Gaussian kernel, $G(x, y, \sigma)$:

$$L(x, y, \sigma) = G(x, y, \sigma) \star f(x, y) \quad (11-66)$$

where the scale is controlled by parameter σ , and G is of the form

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2 + y^2)/2\sigma^2} \quad (11-67)$$

The input image $f(x, y)$ is successively convolved with Gaussian kernels having standard deviations $\sigma, k\sigma, k^2\sigma, k^3\sigma, \dots$ to generate a “stack” of Gaussian-filtered (smoothed) images that are separated by a constant factor k , as shown in the lower left of Fig. 11.56.

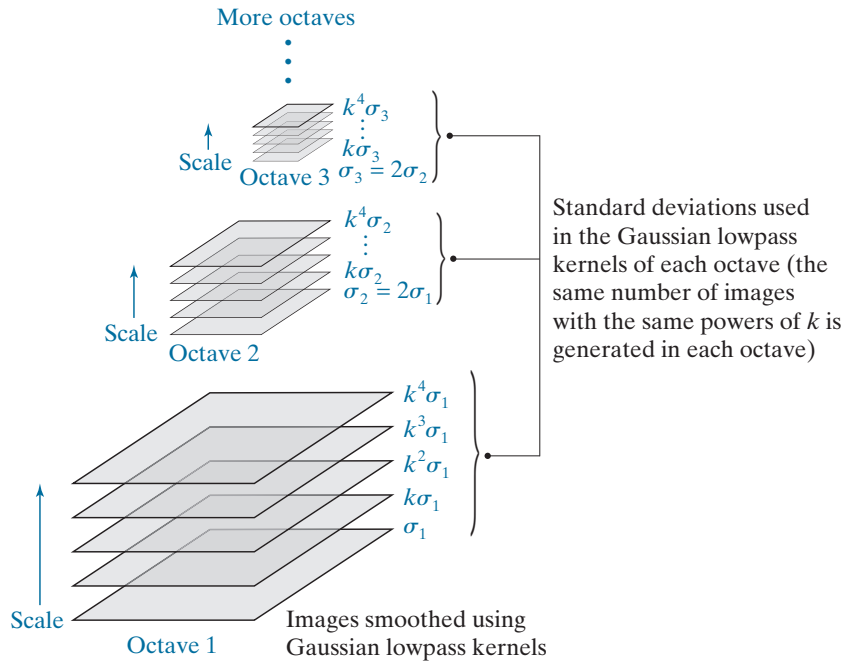
SIFT subdivides scale space into *octaves*, with each octave corresponding to a doubling of σ , just as an octave in music theory corresponds to doubling the frequency of a sound signal. SIFT further subdivides each octave into an integer number, s , of intervals, so that an interval of 1 consists of two images, an interval of 2 consists of three images, and so forth. It then follows that the value used in the Gaussian kernel that generates the image corresponding to an octave is $k^s\sigma = 2\sigma$ which means that $k = 2^{1/s}$. For example, for $s = 2$, $k = \sqrt{2}$, and the input image is successively smoothed using standard deviations of $\sigma, (\sqrt{2})\sigma$, and $(\sqrt{2})^2\sigma$, so that the third image (i.e., the octave image for $s = 2$) in the sequence is filtered using a Gaussian kernel with standard deviation $(\sqrt{2})^2\sigma = 2\sigma$.

[†] Experimental results reported by Lowe [2004] suggest that smoothing the original image using a Gaussian kernel with $\sigma = 0.5$ and then doubling its size by linear (nearest-neighbor) interpolation improves the number of stable features detected by SIFT. This preprocessing step is an integral part of the algorithm. Images are assumed to have values in the range $[0, 1]$.

As in Chapter 3, “ $\star$ ” indicates spatial convolution.

FIGURE 11.56

Scale space, showing three octaves. Because $s = 2$ in this case, each octave has five smoothed images. A Gaussian kernel was used for smoothing, so the space parameter is σ .



The preceding discussion indicates that the number of smoothed images generated in an octave is $s + 1$. However, as you will see in the next section, the smoothed images in scale space are used to compute differences of Gaussians [see Eq. (10-32)] which, in order to cover a full octave, implies that an additional two images past the octave image are required, giving a total of $s + 3$ images. Because the octave image is always the $(s + 1)$ th image in the stack (counting from the bottom), it follows that this image is the third image from the top in the expanded sequence of $s + 3$ images. Each octave in Fig. 11.56 contains five images, indicating that $s = 2$ was used in this case.

The *first* image in the *second* octave is formed by downsampling the original image (by skipping every other row and column), and then smoothing it using a kernel with ~~twice~~ the standard deviation used in the first octave (i.e., $\sigma_2 = 2\sigma_1$). Subsequent images in that octave are smoothed using σ_2 , with the same sequence of values of k as in the first octave (this is denoted by dots in Fig. 11.56). The same basic procedure is then repeated for subsequent octaves. That is, the *first* image of the *new* octave is formed by: (1) downsampling the original image enough times to achieve half the size of the image in the previous octave, and (2) smoothing the downsampled image with a new standard deviation that is twice the standard deviation of the previous octave. The rest of the images in the new octave are obtained by smoothing the downsampled image with the new standard deviation multiplied by the same sequence of values of k as before.

When $k = \sqrt{2}$, we can obtain the first image of a new octave without having to smooth the downsampled image. This is because, for this value of k , the kernel used to smooth the first image of every octave is the same as the kernel used to smooth

Instead of repeatedly downsampling the original image, we can carry the previously downsampled image, and downsample it by 2 to obtain the image required for the next octave.

the *third* image from the top of the previous octave. Thus, the first image of a new octave can be obtained directly by downsampling that third image of the previous octave by 2. The result will be the same (see Problem 11.36). The third image from the top of any octave is called the *octave image* because the standard deviation used to smooth it is twice (i.e., $k^2 = 2$) the value of the standard deviation used to smooth the first image in the octave.

Figure 11.57 uses grayscale images to further illustrate how scale space is constructed in SIFT. Because each octave is composed of five images, it follows that we are again using $s = 2$. We chose $\sigma_1 = \sqrt{2}/2 = 0.707$ and $k = \sqrt{2} = 1.414$ for this example so that the numbers would result in familiar multiples. As in Fig. 11.56, the images going up scale space are blurred by using Gaussian kernels with progressively larger standard deviations, and the first image of the second and subsequent octaves is obtained by downsampling the octave image from the previous octave by 2. As you can see, the images become significantly more blurred (and consequently lose more fine detail) as they go up both in scale as well as in octave. The images in the third octave show significantly fewer details, but their gross appearance is unmistakably that of the same structure.

DETECTING LOCAL EXTREMA

SIFT initially finds the locations of keypoints using the Gaussian filtered images, then refines the locations and validity of those keypoints using two processing steps.

Finding the Initial Keypoints

Keypoint locations in scale space are found initially by SIFT by detecting extrema in the difference of Gaussians of two adjacent scale-space images in an octave, convolved with the input image that corresponds to that octave. For example, to find keypoint locations related to the first two levels of octave 1 in scale space, we look for extrema in the function

$$D(x, y, \sigma) = [G(x, y, k\sigma) - G(x, y, \sigma)] \star f(x, y) \quad (11-68)$$

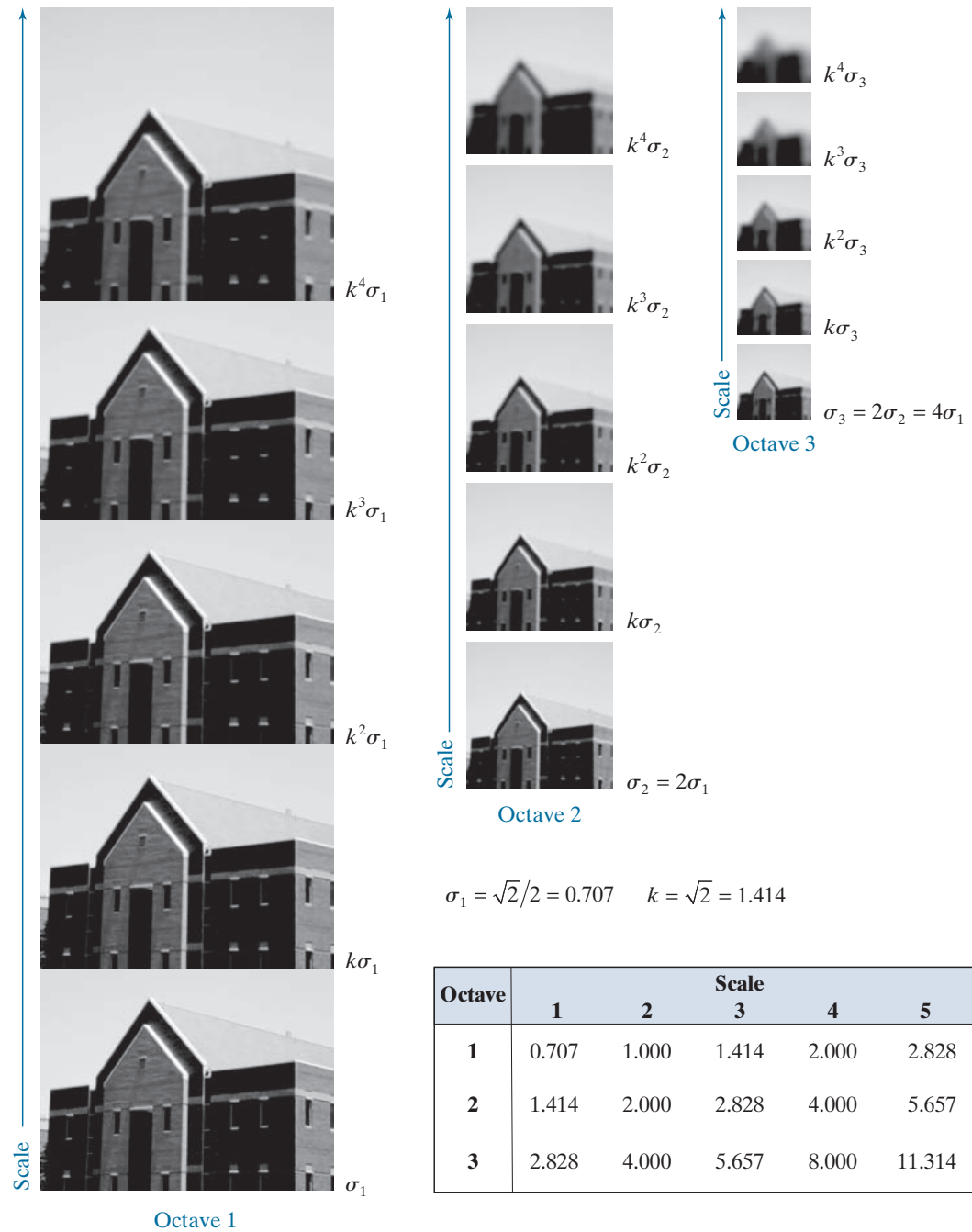
It follows from Eq. (11-66) that

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma) \quad (11-69)$$

In other words, all we have to do to form function $D(x, y, \sigma)$ is subtract the first two images of octave 1. Recall from the discussion of the Marr-Hildreth edge detector (Section 10.2) that the difference of Gaussians is an approximation to the Laplacian of a Gaussian (LoG). Therefore, Eq. (11-69) is nothing more than an approximation to Eq. (10-30). The key difference is that SIFT looks for extrema in $D(x, y, \sigma)$, whereas the Marr-Hildreth detector would look for the zero crossings of this function.

Lindberg [1994] showed that true scale invariance in scale space requires that the LoG be normalized by σ^2 (i.e., that $\sigma^2 \nabla^2 G$ be used). It can be shown (see Problem 11.34) that

FIGURE 11.57 Illustration using images of the first three octaves of scale space in SIFT. The entries in the table are values of standard deviation used at each scale of each octave. For example the standard deviation used in scale 2 of octave 1 is $k\sigma_1$, which is equal to 1.0. (The images of octave 1 are shown slightly overlapped to fit in the figure space.)



$$G(x, y, k\sigma) - G(x, y, \sigma) \approx (k - 1)\sigma^2 \nabla^2 G \tag{11-70}$$

Therefore, DoGs already have the necessary scaling “built in.” The factor $(k - 1)$ is constant over all scales, so it does not influence the process of locating extrema in scale space. Although Eqs. (11-68) and (11-69) are applicable to the first two images

of octave 1, the same form of these equations is applicable to any two images from any octave, provided that the appropriate downsampled image is used, and the DoG is computed from two adjacent images in the octave.

Figure 11.58 illustrates the concepts just discussed, using the building image from Fig. 11.57. A total of $s + 2$ difference functions, $D(x, y, \sigma)$, are formed in each octave from all adjacent pairs of Gaussian-filtered images in that octave. These difference functions can be viewed as images, and one sample of such an image is shown for each of the three octaves in Fig. 11.58. As you might expect from the results in Fig. 11.57, the level of detail in these images decreases the further up we go in scale space.

Figure 11.59 shows the procedure used by SIFT to find extrema in a $D(x, y, \sigma)$ image. At each location (shown in black) in a $D(x, y, \sigma)$ image, the value of the pixel at that location is compared to the values of its eight neighbors in the current image and its nine neighbors in the images above and below. The point is selected as an extremum (maximum or minimum) point if its value is larger than the values of all its neighbors, or smaller than all of them. No extrema can be detected in the first (last) scale of an octave because it has no lower (upper) scale image of the same size.

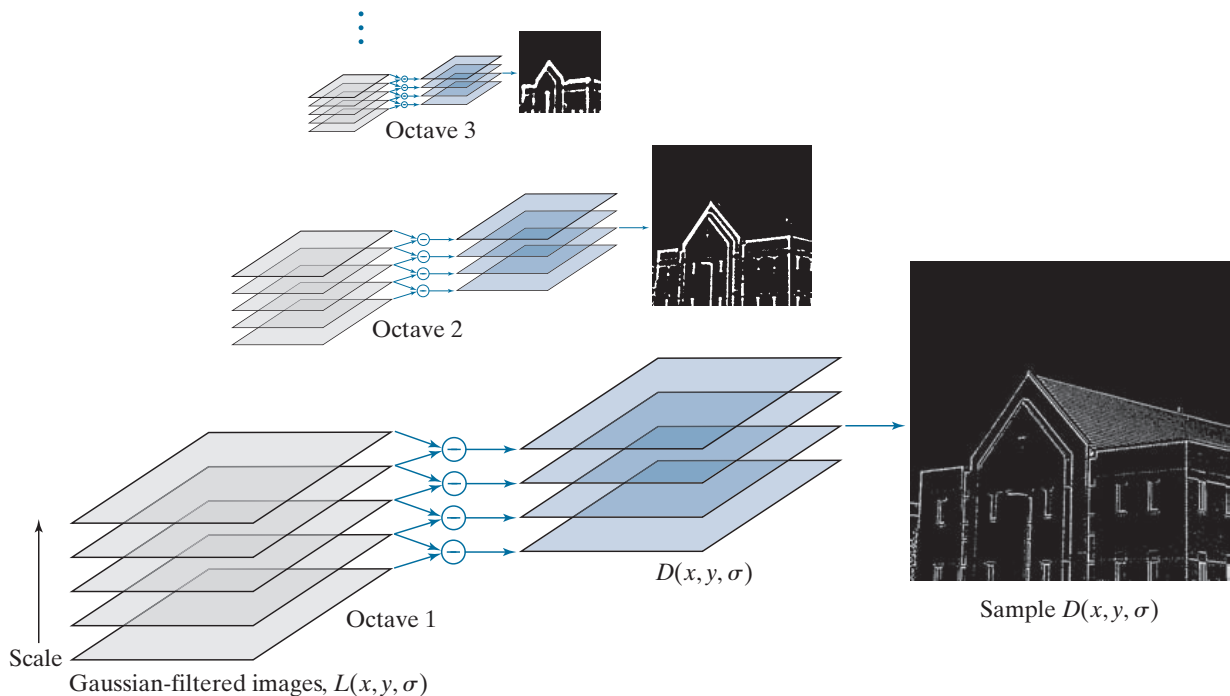
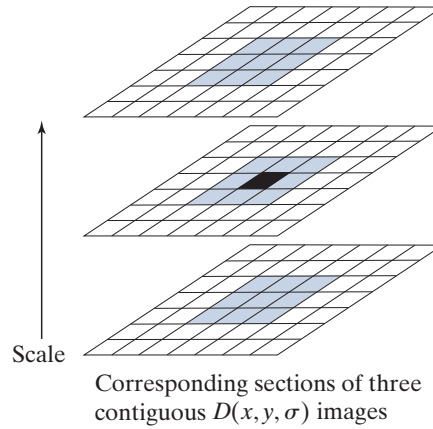


FIGURE 11.58 How Eq. (11-69) is implemented in scale space. There are $s + 3$ $L(x, y, \sigma)$ images and $s + 2$ corresponding $D(x, y, \sigma)$ images in each octave.

FIGURE 11.59

Extrema (maxima or minima) of the $D(x, y, \sigma)$ images in an octave are detected by comparing a pixel (shown in black) to its 26 neighbors (shown shaded) in 3×3 regions at the current and adjacent scale images.



extremum (to achieve subpixel accuracy) is to fit an interpolating function at each extremum point found in the digital function, then look for an improved extremum location in the interpolated function. SIFT uses the linear and quadratic terms of a Taylor series expansion of $D(x, y, \sigma)$, shifted so that the origin is located at the sample point being examined. In vector form, the expression is

$$\begin{aligned} D(\mathbf{x}) &= D + \left(\frac{\partial D}{\partial \mathbf{x}} \right)^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial}{\partial \mathbf{x}} \left(\frac{\partial D}{\partial \mathbf{x}} \right) \mathbf{x} \\ &= D + (\nabla D)^T \mathbf{x} + \frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x} \end{aligned} \quad (11-71)$$

where D and its derivatives are evaluated at the sample point, $\mathbf{x} = (x, y, \sigma)^T$ is the offset from that point, ∇ is the familiar gradient operator,

$$\nabla D = \frac{\partial D}{\partial \mathbf{x}} = \begin{bmatrix} \partial D / \partial x \\ \partial D / \partial y \\ \partial D / \partial \sigma \end{bmatrix} \quad (11-72)$$

and $\mathbf{H}$ is the *Hessian matrix*

$$\mathbf{H} = \begin{bmatrix} \partial^2 D / \partial x^2 & \partial^2 D / \partial x \partial y & \partial^2 D / \partial x \partial \sigma \\ \partial^2 D / \partial y \partial x & \partial^2 D / \partial y^2 & \partial^2 D / \partial y \partial \sigma \\ \partial^2 D / \partial \sigma \partial x & \partial^2 D / \partial \sigma \partial y & \partial^2 D / \partial \sigma^2 \end{bmatrix} \quad (11-73)$$

The location of the extremum, $\hat{\mathbf{x}}$, is found by taking the derivative of Eq. (11-71) with respect to $\mathbf{x}$ and setting it to zero, which gives us (see Problem 11.37):

$$\hat{\mathbf{x}} = -\mathbf{H}^{-1} (\nabla D) \quad (11-74)$$

Because D and its derivatives are evaluated at the sample point, they are constants with respect to $\mathbf{x}$.

The Hessian and gradient of D are approximated using differences of neighboring points, as we did in Section 10.2. The resulting 3×3 system of linear equations is easily solved computationally. If the offset $\hat{\mathbf{x}}$ is greater than 0.5 in any of its three dimensions, we conclude that the extremum lies closer to another sample point, in which case the sample point is changed and the interpolation is performed about that point instead. The final offset $\hat{\mathbf{x}}$ is added to the location of its sample point to obtain the interpolated estimate of the location of the extremum.

The function value at the extremum, $D(\hat{\mathbf{x}})$, is used by SIFT for rejecting unstable extrema with low contrast, where $D(\hat{\mathbf{x}})$ is obtained by substituting Eq. (11-74) into Eq. (11-71), giving (see Problem 11.37):

$$D(\hat{\mathbf{x}}) = D + \frac{1}{2}(\nabla D)^T \hat{\mathbf{x}} \quad (11-75)$$

In the experimental results reported by Lowe [2004], any extrema for which $D(\hat{\mathbf{x}})$ was less than 0.03 was rejected, based on all image values being in the range $[0, 1]$. This eliminates keypoints that have low contrast and/or are poorly localized.

Eliminating Edge Responses

Recall from Section 10.2 that using a difference of Gaussians yields edges in an image. But keypoints of interest in SIFT are “corner-like” features, which are significantly more localized. Thus, intensity transitions caused by edges are eliminated. To quantify the difference between edges and corners, we can look at local curvature. An edge is characterized by high curvature in one direction, and low curvature in the orthogonal direction. Curvature at a point in an image can be estimated from the 2×2 Hessian matrix evaluated at that point. Thus, to estimate local curvature of the DoG at any level in scalar space, we compute the Hessian matrix of D at that level:

$$\mathbf{H} = \begin{bmatrix} \partial^2 D / \partial x^2 & \partial^2 D / \partial x \partial y \\ \partial^2 D / \partial y \partial x & \partial^2 D / \partial y^2 \end{bmatrix} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{yx} & D_{yy} \end{bmatrix} \quad (11-76)$$

where the form on the right uses the same notation as the $\mathbf{A}$ term [Eq. (11-61)] of the Harris matrix (but note that the main diagonals are different). The eigenvalues of $\mathbf{H}$ are proportional to the curvatures of D . As we explained in connection with the Harris-Stephens corner detector, we can avoid direct computation of the eigenvalues by formulating tests based on the trace and determinant of $\mathbf{H}$, which are equal to the sum and product of the eigenvalues, respectively. To use notation different from the HS discussion, let α and β be the eigenvalues of $\mathbf{H}$ with the largest and smallest magnitude, respectively. Using the relationship between the eigenvalues of $\mathbf{H}$ and its trace and determinant we have (remember, $\mathbf{H}$ is symmetric and of size 2×2):

$$\begin{aligned} \text{Tr}(\mathbf{H}) &= D_{xx} + D_{yy} = \alpha + \beta \\ \text{Det}(\mathbf{H}) &= D_{xx}D_{yy} - (D_{xy})^2 = \alpha\beta \end{aligned} \quad (11-77)$$

If you display an image as a topographic map (see Fig. 2.18), edges will appear as ridges that have low curvature along the ridge and high curvature perpendicular to it.

As with the HS corner detector, the advantage of this formulation is that the trace and determinants of 2×2 matrix $\mathbf{H}$ are easy to compute. See the margin note in Eq. (11-63).

If the determinant is negative, the curvatures have different signs and the keypoint in question cannot be an extremum, so it is discarded.

Let r denote the ratio of the largest to the smallest eigenvalue. Then $\alpha = r\beta$ and

$$\frac{[\text{Tr}(\mathbf{H})]^2}{\text{Det}(\mathbf{H})} = \frac{(\alpha + \beta)^2}{\alpha\beta} = \frac{(r\beta + \beta)^2}{r\beta^2} = \frac{(r + 1)^2}{r} \quad (11-78)$$

which depends on the ratio of the eigenvalues, rather than their individual values. The minimum of $(r + 1)^2/r$ occurs when the eigenvalues are equal, and it increases with r . Therefore, to check that the ratio of principal curvatures is below some threshold, r , we only need to check

$$\frac{[\text{Tr}(\mathbf{H})]^2}{\text{Det}(\mathbf{H})} < \frac{(r + 1)^2}{r} \quad (11-79)$$

which is a simple computation. In the experimental results reported by Lowe [2004], a value of $r = 10$ was used, meaning that keypoints with ratios of curvature greater than 10 were eliminated.

Figure 11.60 shows the SIFT keypoints detected in the building image using the approach discussed in this section. Keypoints for which $D(\hat{\mathbf{x}})$ in Eq. (11-75) was less than 0.03 were rejected, as were keypoints that failed to satisfy Eq. (11-79) with $r = 10$.

KEYPOINT ORIENTATION

At this point in the process, we have computed keypoints that SIFT considers stable. Because we know the location of each keypoint in scale space, we have achieved *scale independence*. The next step is to assign a consistent orientation to each keypoint based on local image properties. This allows us to represent a keypoint relative to its orientation and thus achieve *invariance to image rotation*. SIFT uses a

FIGURE 11.60
SIFT keypoints detected in the building image. The points were enlarged slightly to make them easier to see.



See Section 10.2 regarding computation of the gradient magnitude and angle.

straightforward approach for this. The scale of the keypoint is used to select the Gaussian smoothed image, L , that is closest to that scale. In this way, all orientation computations are performed in a scale-invariant manner. Then, for each image sample, $L(x, y)$, at this scale, we compute the gradient magnitude, $M(x, y)$, and orientation angle, $\theta(x, y)$, using pixel differences:

$$M(x, y) = \left[(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2 \right]^{\frac{1}{2}} \quad (11-80)$$

and

$$\theta(x, y) = \tan^{-1} \left[(L(x, y+1) - L(x, y-1)) / (L(x+1, y) - L(x-1, y)) \right] \quad (11-81)$$

A histogram of orientations is formed from the gradient orientations of sample points in a neighborhood of each keypoint. The histogram has 36 bins covering the 360° range of orientations on the image plane. Each sample added to the histogram is weighed by its gradient magnitude, and by a circular Gaussian function with a standard deviation 1.5 times the scale of the keypoint.

Peaks in the histogram correspond to dominant local directions of local gradients. The highest peak in the histogram is detected and any other local peak that is within 80% of the highest peak is used also to create another keypoint with that orientation. Thus, for the locations with multiple peaks of similar magnitude, there will be multiple keypoints created at the same location and scale, but with *different* orientations. SIFT assigns multiple orientations to only about 15% of points with multiple orientations, but these contribute significant to image matching (to be discussed later and in Chapter 12). Finally, a parabola is fit to the three histogram values closest to each peak to interpolate the peak position for better accuracy.

Figure 11.61 shows the same keypoints as Fig. 11.60 superimposed on the image and showing keypoint orientations as arrows. Note the consistency of orientation

FIGURE 11.61

The keypoints from Fig. 11.60 superimposed on the original image. The arrows indicate keypoint orientations.



of similar sets of keypoints in the image. For example, observe the keypoints on the right, vertical corner of the building. The lengths of the arrows vary, depending on illumination and image content, but their direction is unmistakably consistent. Plots of keypoint orientations generally are quite cluttered and are not intended for general human interpretation. The value of keypoint orientation is in image matching, as we will illustrate later in our discussion.

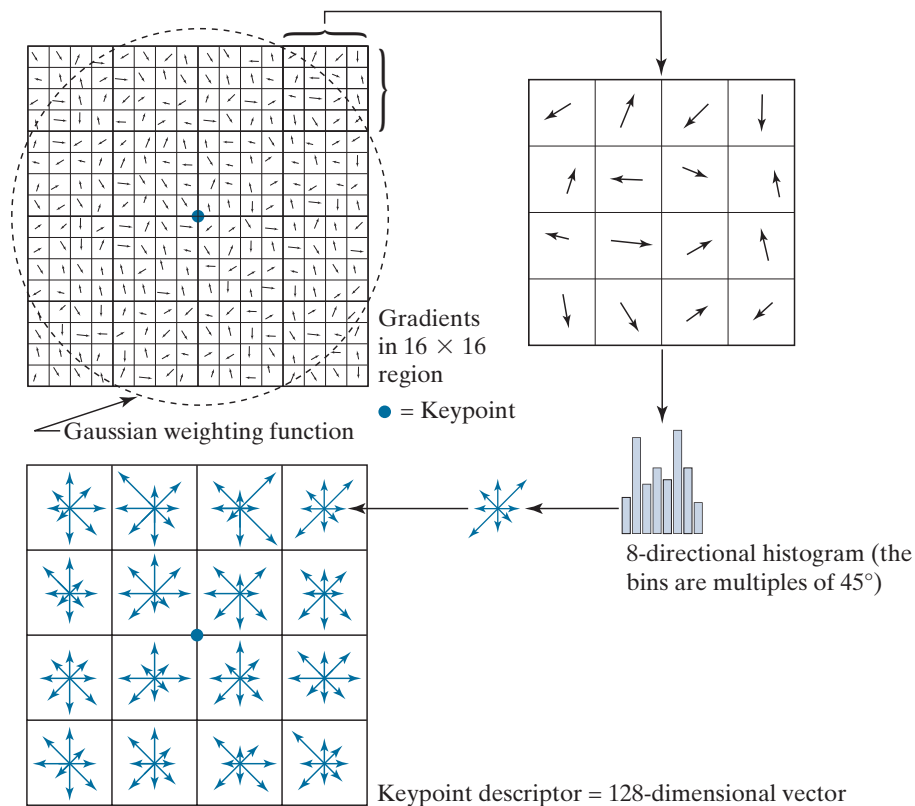
KEYPOINT DESCRIPTORS

The procedures discussed up to this point are used for assigning an image location, scale, and orientation to each keypoint, thus providing invariance to these three variables. The next step is to compute a descriptor for a local region around each keypoint that is highly distinctive, but is at the same time as invariant as possible to changes in scale, orientation, illumination, and image viewpoint. The idea is to be able to use these descriptors to identify matches (similarities) between local regions in two or more images.

The approach used by SIFT to compute descriptors is based on experimental results suggesting that local image gradients appear to perform a function similar to what human vision does for matching and recognizing 3-D objects from different viewpoints (Lowe [2004]). Figure 11.62 summarizes the procedure used by SIFT to generate the descriptors associated with each keypoint. A region of size 16×16

FIGURE 11.62

Approach used to compute a keypoint descriptor.



pixels is centered on a keypoint, and the gradient magnitude and direction are computed at each point in the region using pixel differences. These are shown as randomly oriented arrows in the upper-left of the figure. A Gaussian weighting function with standard deviation equal to one-half the size of the region is then used to assign a weight that multiplies the magnitude of the gradient at each point. The Gaussian weighting function is shown as a circle in the figure, but it is understood that it is a bell-shaped surface whose values (weights) decrease as a function of distance from the center. The purpose of this function is to reduce sudden changes in the descriptor with small changes in the position of the function.

Because there is one gradient computation for each point in the region surrounding a keypoint, there are $(16)^2$ gradient directions to process for each keypoint. There are 16 directions in each 4×4 subregion. The top-rightmost subregion is shown zoomed in the figure to simplify the explanation of the next step, which consists of quantizing all gradient orientations in the 4×4 subregion into eight possible directions differing by 45° . Rather than assigning a directional value as a full count to the bin to which it is closest, SIFT performs interpolation that *distributes* a histogram entry among *all* bins proportionally, depending on the distance from that value to the center of each bin. This is done by multiplying each entry into a bin by a weight of $1 - d$, where d is the shortest distance from the value to the center of a bin, measured in the units of the histogram spacing, so that the maximum possible distance is 1. For example, the center of the first bin is at $45^\circ/2 = 22.5^\circ$, the next center is at $22.5^\circ + 45^\circ = 67.5^\circ$, and so on. Suppose that a particular directional value is 22.5° . The distance from that value to the center of the first histogram bin is 0, so we would assign a full entry (i.e., a count of 1) to that bin in the histogram. The distance to the next center would be greater than 0, so we would assign a fraction of a full entry, that is $1 * (1 - d)$, to that bin, and so forth for all bins. In this way, every bin gets a proportional fraction of a count, thus avoiding “boundary” effects in which a descriptor changes abruptly as a small change in orientation causes it to be assigned from one bin to another.

Figure 11.62 shows the eight directions of a histogram as a small cluster of vectors, with the length of each vector being equal to the value of its corresponding bin. Sixteen histograms are computed, one for each 4×4 subregion of the 16×16 region surrounding a keypoint. A descriptor, shown on the lower left of the figure, then consists of a 4×4 array, each containing eight directional values. In SIFT, this descriptor data is organized as a 128-dimensional vector.

In order to achieve orientation invariance, the coordinates of the descriptor and the gradient orientations are rotated relative to the keypoint orientation. **In order to reduce the effects of illumination, a feature vector is normalized in two stages.** First, the vector is normalized to unit length by dividing each component by the vector norm. A change in image contrast resulting from each pixel value being multiplied by a constant will multiply the gradients by the same constant, so the change in contrast will be cancelled by the first normalization. A brightness change caused by a constant being added to each pixel will not affect the gradient values because they are computed from pixel differences. **Therefore, the descriptor is invariant to affine changes in illumination.** However, nonlinear illumination changes resulting, for example, from camera saturation, can also occur. These types of changes can cause large variations in the relative magnitudes of some of the gradients, but they

are less likely to affect gradient orientation. SIFT reduces the influence of large gradient magnitudes by thresholding the values of the normalized feature vector so that all components are below the experimentally determined value of 0.2. After thresholding, the feature vector is renormalized to unit length.

SUMMARY OF THE SIFT ALGORITHM

As the material in the preceding sections shows, SIFT is a complex procedure consisting of many parts and empirically determined constants. The following is a step-by-step summary of the method.

As indicated at the beginning of this section, smoothing and doubling the size of the input image is assumed. Input images are assumed to have values in the range $[0, 1]$.

1. **Construct the scale space.** This is done using the procedure outlined in Figs. 11.56 and 11.57. The parameters that need to be specified are σ , s , (k is computed from s), and the number of octaves. Suggested values are $\sigma = 1.6$, $s = 2$, and three octaves.
2. **Obtain the initial keypoints.** Compute the difference of Gaussians, $D(x, y, \sigma)$, from the smoothed images in scale space, as explained in Fig. 11.58 and Eq. (11-69). Find the extrema in each $D(x, y, \sigma)$ image using the method explained in Fig. 11.59. These are the initial keypoints.
3. **Improve the accuracy of the location of the keypoints.** Interpolate the values of $D(x, y, \sigma)$ via a Taylor expansion. The improved key point locations are given by Eq. (11-74).
4. **Delete unsuitable keypoints.** Eliminate keypoints that have low contrast and/or are poorly localized. This is done by evaluating D from Step 3 at the improved locations, using Eq. (11-75). All keypoints whose values of D are lower than a threshold are deleted. A suggested threshold value is 0.03. Keypoints associated with edges are deleted also, using Eq. (11-79). A value of 10 is suggested for r .
5. **Compute keypoint orientations.** Use Eqs. (11-80) and (11-81) to compute the magnitude and orientation of each keypoint using the histogram-based procedure discussed in connection with these equations.
6. **Compute keypoint descriptors.** Use the method summarized in Fig. 11.62 to compute a feature (descriptor) vector for each keypoint. If a region of size 16×16 around each keypoint is used, the result will be a 128-dimensional feature vector for each keypoint.

The following example illustrates the power of this algorithm.

EXAMPLE 11.20: Using SIFT for image matching.

We illustrate the performance of the SIFT algorithm by using it to find the number of matches between an image of a building and a subimage formed by extracting part of the right corner edge of the building. We also show results for rotated and scaled-down versions of the image and subimage. This type of process can be used in applications such as finding correspondences between two images for the purpose of image registration, and for finding instances of an image in a database of images.

Figure 11.63(a) shows the keypoints for the building image (this is the same as Fig. 11.61), and the keypoints for the subimage, which is a separate, much smaller image. The keypoints were computed

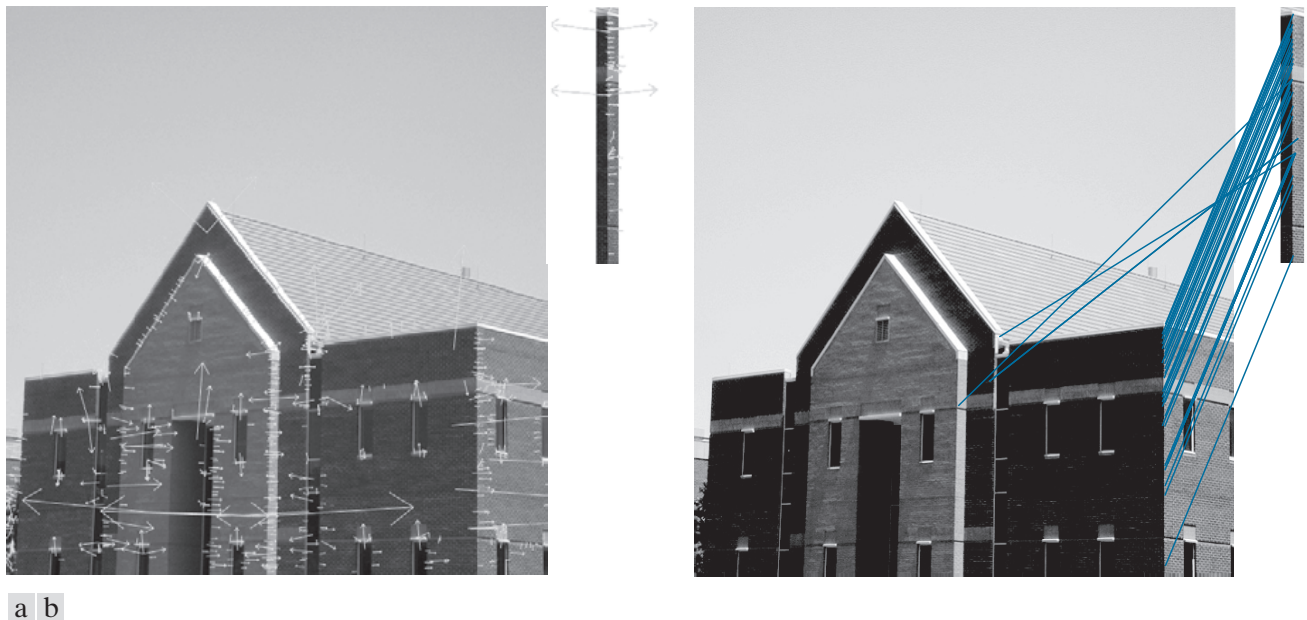


FIGURE 11.63 (a) Keypoints and their directions (shown as gray arrows) for the building image and for a section of the right corner of the building. The subimage is a separate image and was processed as such. (b) Corresponding key points between the building and the subimage (the straight lines shown connect pairs of matching points). Only three of the 36 matches found are incorrect.

using SIFT *independently* for each image. The building shows 643 keypoints and the subimage 54 keypoints. Figure 11.63(b) shows the matches found by SIFT between the image and subimage; 36 keypoint matches were found and, as the figure shows, only three were incorrect. Considering the large number of initial keypoints, you can see that keypoint descriptors offer a high degree of accuracy for establishing correspondences between images.

Figure 11.64(a) shows keypoints for the building image after it was rotated by 5° counterclockwise, and for a subimage extracted from its right corner edge. The rotated image is smaller than the original because it was cropped to eliminate the constant areas created by rotation (see Fig. 2.41). Here, SIFT found 547 keypoints for the building and 49 for the subimage. A total of 26 matches were found and, as Fig. 11.64(b) shows, only two were incorrect.

Figure 11.65 shows the results obtained using SIFT on an image of the building reduced to half the size in both spatial directions. When SIFT was applied to the downsampled image and a corresponding subimage, no matches were found. This was remedied by brightening the reduced image slightly by manipulating the intensity gamma. The subimage was extracted from this image. Despite the fact that SIFT has the capability to handle some degree of changes in intensity, this example indicates that performance can be improved by enhancing the contrast of an image prior to processing. When working with a database of images, histogram specification (see Chapter 3) is an excellent tool for normalizing the intensity of all images using the characteristics of the image being queried. SIFT found 195 keypoints for the half-size image and 24 keypoints for the corresponding subimage. A total of seven matches were found between the two images, of which only one was incorrect.

The preceding two figures illustrate the insensitivity of SIFT to rotation and scale changes, but they are not ideal tests because the reason for seeking insensitivity to these variables in the first place is